Semantic Similarity Search Service

By Miguel Nogueira

Introduction

Most consumers accept online Terms of Service (ToS) contracts without reading them, creating a severe informational asymmetry that online platforms frequently exploit by hiding legally unfair or detrimental clauses. While European consumer law prohibits these practices, consumer protection agencies lack the labour and resources to manually audit thousands of text-dense documents. To solve this bottleneck, this project builds a containerized, production-ready backend service that utilizes semantic search and machine learning (ML) classification, trained on the CLAUDETTE corpus dataset (Lippi et al, 2019), to automatically detect and categorize potentially unfair clauses at a sentence level without relying on large language models (LLMs).

Literature review (in a nutshell)

Lippi et al. (2019) employed ML classification to automatically detect potentially unfair clauses in the CLAUDETTE dataset. They employed bag-of-words (BoW) using a Term Frequency - Inverse Document Frequency (TF-IDF) approach to encode the sentences, which was shown to achieve comparatively better performances than other more sophisticated approaches. Their results further showed the best-performing classification system was an ensemble method that utilized a majority voting strategy across five distinct machine learning models: a single lexical SVM, an array of eight category-specific SVMs, a syntactic Tree Kernel model, and two structured SVM-HMM sequence models, ultimately achieving the highest overall macro-averaged F1 score of 0.806.

Recent research has significantly advanced beyond traditional BoW by utilizing pre-trained language models and semantic text embeddings. Specifically, Akash et al. (2024) showed that transformer-based embedding (specifically Legal-BERT) applied to the same CLAUDETTE dataset model dramatically outperforms traditional TF-IDF approaches. By pairing these domain-specific legal text embeddings with a Support Vector Classifier (SVC), their architecture successfully captured deep semantic nuances within complex contractual language, achieving a state-of-the-art macro-averaged F1 score of 0.922 for the binary clause detection task, representing a substantial performance leap over the 0.806 ceiling set by the original paper's complex ensemble method.

Data & Methods

Datasets

The present work focuses on the CLAUDETTE corpus dataset, published by Lippi et al. (2019). CLAUDETTE corpus is a dataset of 50 annotated online consumer contracts, to automate the detection and categorization of potentially unfair clauses using standard machine learning models.

Approach to the problem & key design decisions

The solution has two main components:

1) Indexing and semantic search
2) Classification component to detect "unfair clauses"

Below we provide a more detailed explanation of the implementation of each component. But first, I'd like to highlight that the specific decisions on the algorithms used came from experience and 3 key points from the (quick) literature review:

1) Akash et al. (2024) demonstrated that LEGAL-BERT was superior to TF-IDF (employed by Lippi et al., 2019) for binary unfair classification and unfair clause tagging in the CLAUDETTE dataset. Hence, we assumed LEGAL-BERT to be the state-of-the-art (SOTA) and dropped TF-IDF as the benchmark. The experimental question then became "is LEGAL-BERT better than simpler embedding methods?". To answer this question we compared LEGAL-BERT to all-MiniLM-L6-v2.
2) Akash et al. (2024) also showed that Support Vector Machines with a Radial Basis Function kernel (SVC RBF) combined with LEGAL-BERT obtained the best performance of all methods tested. The Multi-Layer Perceptron (MLPClassifier) also showed good performance for this classification task. However, there are many other classification models available that were not considered, including the widely used and versatile gradient boosting and random forest algorithms, but also the very efficient and interpretable Logistic Regression. Consequently, here it was decided to experiment with a wider variety of classification models.
3) Lippi et al (2019) had suggested that an ensemble of multiple classifiers was superior to all individual models. Akash et al. (2024) did not consider ensemble setups.

Why the focus on literature, specifically recent literature? The answer is simple, credibility and defensibility. You are doing something in the legal space so your clients/users may ask: why did you use this methodology? Why should I trust your models? Part of the answer “our methodology is grounded on recent scientific research”.

Methodology

Indexing and semantic search

The process of indexing a corpus of sentences transforms unstructured raw text into a structured, highly optimized numerical format suitable for production environments.

First the CLAUDETTE txt files of sentences+labels were combined into a pandas dataframe. Subsequently, the indexing process converts unstructured text dataframes into structured, highly optimized formats tailored to the capacity of the chosen embedding model.

We experimented with 2 different solutions:

- The general-purpose SimpleEmbeddingEngine initialized with all-MiniLM-L6-v2, the unified transformation pipeline encodes each text sequence into a 384-dimensional dense feature matrix.
- the specialized LegalBertEmbeddingEngine scales the output vector space to a 768-dimensional matrix, leveraging a domain-specific architecture optimized for legal terminology.

To avoid the computational overhead of re-encoding the entire dataset upon every service restart, these generated float32 mathematical arrays are serialized directly to disk as binary numpy structures. Simultaneously, the corresponding textual metadata, including structural tracking fields like company identifiers, sentence indices, and classification flags, is preserved as a serialized pickle lookup table to create an instant-load mechanism that achieves complete persistence.

To support semantic similarity search, both implementations rely on spatial proximity queries calculated within their respective continuous feature spaces. When a raw string query enters the system, it passes through the exact same transformation logic as the original corpus, converting the text into a matching 384-dimensional or 768-dimensional vector representation depending on whether all-MiniLM-L6-v2 or Legal-BERT is active.

The service then executes a matrix operations step, applying cosine similarity algorithms between the newly generated query vector and the pre-loaded corpus matrix to evaluate their contextual alignment. Cosine distance is the

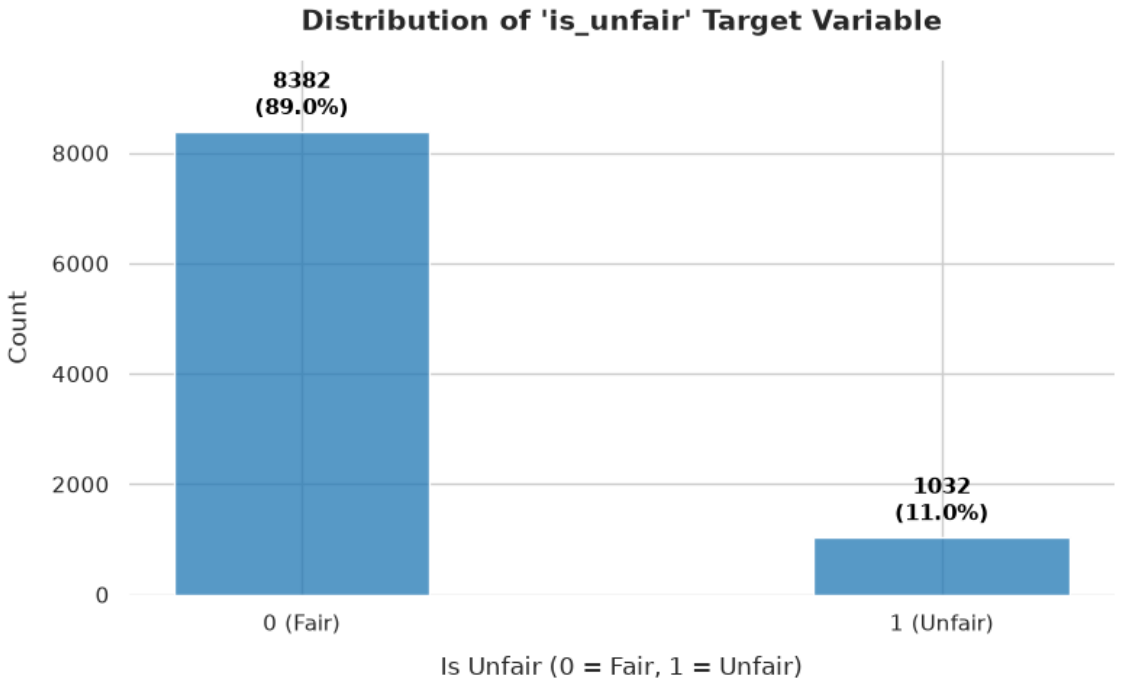
ideal choice because it evaluates semantic similarity based purely on the directional angle of the vectors, ensuring that short user queries and longer corpus sentences are matched on contextual meaning rather than text length. Furthermore, when vectors are unit-normalized, the calculation simplifies to a highly optimized dot product, minimizing computational overhead during production matrix operations. The resulting similarity scores are sorted in descending order to isolate the highest-ranking indices, allowing the system to map those mathematical coordinates back to the metadata lookup table and output a precisely ordered list of contextually relevant results.

Classification component to detect "unfair clauses"

To automatically detect unfair terms of service provisions, the architecture integrates a modular classification layer that operates directly over the generated dense semantic embedding spaces.

To systematically determine the optimal predictive model for the legal domain, we constructed an automated experimentation framework evaluating six diverse estimator families: Logistic Regression, Linear Support Vector Classification (LinearSVC), Support Vector Machines with a Radial Basis Function kernel (SVC RBF), Random Forest, Histogram-Based Gradient Boosting, and a multi-layer Perceptron (MLP Neural Network).

Because contract datasets exhibit severe class imbalances where unfair clauses represent a small minority, estimators are injected with class-weight balancing hyper-parameters where supported, or preceded by robust standardization pipelines (StandardScaler) to prevent numerical dominance in non-linear spaces. For Histogram-Based Gradient Boosting which lacks native string-based weight balancing, the model relies on gradient-based iterative updates over the raw sample distribution.



The optimization routine isolates an independent evaluation split through a 80-20 train-test division, with a controlled random seed for reproducibility.

Hyperparameter optimization is achieved via a localized RandomizedSearchCV executing a 5-fold stratified cross. The tuning engine prioritizes the macro-averaged F1-score as its primary optimization metric following the literature reviewed articles, ensuring equal optimization weight is applied to both fair and unfair clause boundaries.

Upon identifying the optimal structural parameters within the target grid space, the underlying wrapper freezes the top-performing estimator configuration. To enforce persistence and eliminate real-time training overhead, the finalized pipeline is serialized directly to disk as a frozen binary structure via joblib. In production, when an incoming query vector enters the service, it bypasses iterative fitting loops and is routed straight to the restored model instance to output a deterministic classification flag.

To quantify prediction errors and evaluate model generalization, the pipeline extracts a multidimensional suite of evaluation metrics from the independent test partition. Because the classification layer applies standard decision thresholds over the models underlying continuous probability mappings, performance is assessed using the hard binary discrete predictions against the ground-truth annotations. The framework evaluates the overall performance topology using accuracy, macro-averaged precision, macro-averaged recall, and the macro-averaged F1-score derived from the test confusion matrix. While overall accuracy provides a baseline metric for total correct predictions, it is highly sensitive to class skew and remains an unreliable indicator in imbalanced contract datasets. Consequently, error types are decoupled through precision

and recall to penalize false positives and false negatives, capturing both incorrect compliance violations and failures to isolate genuinely unfair provisions. The macro F1-score then serves as the primary optimization metric by calculating the harmonic mean of precision and recall across both class boundaries independently, providing an unweighted global average that directly penalizes any algorithmic bias toward the dominant fair clause distribution.

Following Lippi et al. (2019) the architecture incorporates a final meta-ensemble layer utilizing a stacking configuration. The framework extracts the pre-tuned weights of the frozen base estimators from disk and aggregates them into a unified stacking pipeline. Rather than passing raw semantic embedding vectors directly to the final decision layer, the system restricts the input space exclusively to the prediction vectors generated by the base estimators. To prevent data leakage and overfitting during the ensemble stage, a meta-learner is optimized using a 5-fold internal cross-validation routing to generate out-of-fold prediction matrices. A balanced logistic regression model serves as the final meta-estimator, learning to mathematically weight and reconcile the divergent decisions of the base pipelines. The resulting meta-ensemble is evaluated against the identical stratified validation partition and frozen to disk via joblib, providing a highly generalized production pipeline optimized for complex legal compliance boundaries.

To further fortify real-time inference against the localized distribution shifts common in heterogeneous terms of service agreements, the architecture appends an empirical Bayesian post-processing layer to the classification pipeline. Rather than relying solely on the frozen estimator's isolated probability output, the system dynamically updates this prior probability using contextual evidence harvested from a semantic search neighborhood. Notice that the additional burden to the operational API is relatively small because the Bayesian inference postprocessing relies on ML classification output and semantic search results, both available in the API call.

The system queries the vector space to extract its top nearest neighbors, filtering out irrelevant matches via a strict cosine similarity floor >0.5. To heavily reward ultra-close semantic matches, these neighbour votes are heavily scaled by squaring their similarity scores before computing a localized ratio of unfairness. This local ratio is then evaluated against the global corpus base-rate, with a value 11% to reflect the historical distribution of the CLAUDETTE corpus, to generate an empirical likelihood ratio. By multiplying the model’s prior odds by this likelihood ratio, the architecture dynamically shifts the final decision boundary: if a clause lands in a dense cluster of known unfair provisions, its posterior probability is scaled upward; conversely, if the neighbourhood is entirely pristine, the probability is adjusted downward. If the semantic search engine returns zero valid neighbours, the system bypasses the Bayesian

update entirely, smoothly reverting to the model's unadjusted prior probability to ensure deterministic stability.

The table below shows that the combination of Legal-BERT with SVC-RBF achieves the best overall performance (and specifically the best Macro F1). While this result not very interesting academically because it just matches the results from Akash et al. (2024), from the operational point of view that is actually good, because the decision follow the previous research grounded setup is highly defensible and credible as mentioned above. The results further demonstrate the added value of the Bayesian postprocessing step in enhancing the classification performance on top of the 2 best configurations (Legal-BERT with SVC-RBF and Legal-BERT with MLPClassifier) results in no significant model performance gains (it may even degrade slightly the model in some cases). Notice that the Bayesian postprocessing was not tested for all configurations due to lack of time for further testing. And in your head you are asking “Miguel, but if it didn’t improve why did you keep the Bayesian mumbo jumbo?” Great question. Well, improving models beyond already optimized (previously researched) performance is hard and difficult to get at first shot. But the Bayesian idea is actually interesting, if we have a clause that is nearly identical to previous clauses that we have already carefully labelled, shouldn’t this information affect our prediction, especially considering we already have all the ingredients (initial probability and semantic search similarity measure of top matches)? Conceptually yes, in practice, maybe. Just because it didn’t work in 30 minutes of research doesn’t mean we should drop it completely, but also just because it sounds good doesn’t mean it’s not a waste of time pursuing it. So let’s call it backlog and think about it in the future. Check out what has been done in this space by other people and alternative solutions out there... and then decide.

Until then, we keep our research grounded Legal-BERT with SVC-RBF (no Bayesian on top) configuration as the default configuration for the API

Model	Macro F1	Precision	Recall	Accuracy
SimpleEmbeddingEngine				
LogisticRegression	0.727416	0.691178	0.842342	0.848115
LinearSVC	0.731772	0.694674	0.846260	0.851301
SVC_RBF_Pipeline	0.854551	0.864273	0.845508	0.944769
RandomForest	0.752610	0.754814	0.750462	0.904408
HistGradientBoosting	0.714960	0.855354	0.664238	0.917685
MLPClassifier	0.835156	0.874236	0.805397	0.941583

MetaEnsemble_Stacking	0.804343	0.757673	0.901963	0.901221
LegalBertEmbeddingEngine				
LogisticRegression	0.819640	0.779893	0.884763	0.916091
LinearSVC	0.817844	0.771845	0.905201	0.910781
SVC_RBF_Pipeline	**0.871216**	**0.901986**	**0.846021**	**0.953266**
RandomForest	0.799498	0.865318	0.758046	0.933086
HistGradientBoosting	0.797392	0.896392	0.744633	0.935741
MLPClassifier	0.859206	0.880421	0.840910	0.947955
MetaEnsemble_Stacking	0.844179	0.798451	0.922363	0.926182
SVC_RBF + Bayesian Post.	0.865789	0.835440	0.905759	0.942113
MLPClassifier + Bayesian	0.870743	0.842300	0.907250	0.944770

Trade-offs and limitations

Other algorithms for ML classification, enconding and similarity search are available out there. The present choices were informed by a very quick literature review and past experience. The goal here was to highlight how I would implement this more than have the perfect solution. Perfect solutions don't exist, and if they did they would take a lot more time, research and team work to get.

Model choice could have been different. As explained above this specific choices were guided by 1) quick SOTA literature review and 2) personal experience. The code (core/classifier.py) was built in such a way that it is quite easy to add or remove new models just by editing the global variables MODELS_TO_EXPERIMENT & TUNING_GRIDS on top of the script, while keeping their current simple and intuitive structure.

Model training takes a while. However, given the experimental nature of this training comparing 2 embedding approaches and multiple classification models (with respective tuning of hyper-parameters) was considered to be relevant here for tech task demo purposes. Furthermore, since model training is done offline the longer training time was deemed acceptable here.

Care was taken to provide the trained models with the delivered repo, so that the evaluator does not require to waste her/his time (but the code and instructions to run the training are also provided)

Some combinations in the model hyperparameter space are not allowed (e.g., for logistic regression solver lbfgs doesn't support 'l1' penalty. The Random Search implementation just skips through these cases without breaking the code, so this brute force kind of approach was tolerated.

There is trade-off on interpretability when using models like SVC-RBF compared to logistic regression, but I think that the output of semantic search (similar entries) and classification as potentially unfair is interpretable enough that users don't necessarily need to understand how the model gets there, especially considering that the methods to get there are based on previous research.

In feature_engine.py an abstraction could be built to allow for different models (e.g. all-MiniLM-L6-v2 and legal-bert). However building this abstraction was considered to be a waste of time because we had only a few hours to develop the full pipeline and the goal was to choose the best method.

Didn't have time to further refactor the code nor implement unit tests. In the available time I had, either delivered the full task or partial task with unit tests and more refactoring…went the former.

I found a more recent paper Emmanuel et al. (2025) where they report superior performance by moving beyond static feature extraction to end-to-end fine-tuning, allowing the internal weights of transformer models like LEGAL-BERT and DistilBERT to adapt dynamically to the semantic definition of "unfairness" in the CLAUDETTE corpus. While their approach seems promising, it was something not very familiar to me and I have decided to stay with the ML classifier methodology. However, with a little more time (that the few hours I had available for this technical task, it would be interesting to experiment with this approach.

References

Akash, B. S., Kupireddy, A., & Murthy, L. B. (2024). Unfair TOS: An automated approach using customized BERT. *arXiv preprint arXiv:2401.11207*. https://arxiv.org/abs/2401.11207

Emmanuel, J. T., Soeseno, S., Chandra, B. A. X. H., & Lucky, H. (2025). Domain-Aware Classification of Terms of Service Clause Fairness Using BERT. In *2025 5th International Conference of Science and Information Technology in Smart Administration (ICSINTESA)* (pp. 719-724). IEEE. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=11413795

Lippi, M., Pałka, P., Contissa, G., Lagioia, F., Micklitz, H. W., Sartor, G., & Torroni, P. (2019). CLAUDETTE: An automated detector of potentially unfair clauses in online terms of service. *Artificial Intelligence and Law*, *27*(2), 117–139. https://doi.org/10.1007/s10506-019-09243-2